Manifold Constraint Hyper Connection (MHC)

Replaces the standard residual skip connection $x_{l+1} = F(x_l) + x_l$ with a structured "hyper-connection" that mixes and routes multiple embedding lanes.

Motivation: residual skips help gradients flow (since $x'_{l+1} = F'(x_l) + I$). MHC generalizes this by (1) projecting multiple lanes into a smaller compute lane, (2) applying a heavy block (MoE/Attention) there, and (3) redistributing the result back to the full lane dimension.

Standard hyper-connection

The HC layer is written compactly as

$$X_{l+1} = B_l X_l + C_l \, F_l\big(A_l X_l\big)$$

where the terms are described below.

Components

- X_l, X_{l+1}: input and output of layer l respectively, where $X_l \in R^{n_{hc} \times d}$.
- n_{hc}: the number of hyper-connection lanes.
- d: the embedding width.
- A_l: projection that squeezes the n_{hc} lanes into a single lane. Shape: $A_l \in R^{1 \times n_{hc}}$, so $A_l X_l \in R^{1 \times d}$.
- $F_l(\cdot)$: the expensive compute block (e.g. MoE or multi-head attention) that operates on the compressed $R^{1 \times d}$ lane and returns $R^{1 \times d}$.
- C_l: expansion (or coloring) matrix that maps the $(1, d)$ output of F_l back to (n_{hc}, d). Shape: $C_l \in R^{n_{hc} \times 1}$, so $C_l \, F_l(A_l X_l) \in R^{n_{hc} \times d}$.
- B_l: mixes the original n_{hc} input lanes. Shape: $B_l \in R^{n_{hc} \times n_{hc}}$; input and output retain shape $X_l, B_l X_l \in R^{n_{hc} \times d}$ before adding to the expanded path, creating the bypass connection.

Why

- we make a massive dimension d = 7168, each value in this vector is a feature of the token
- we use multi-head attention, each head in the 7168 embedding tries to learn a single group of features (grammer, semantics, context), and they fuse at the end using some kind of projection. After a lot of $+x$ operations, the output stiffens (100 layers of operations meld into 1 brown turd)
- we use multiple channels (eg. 4), each with a full 7168 embedding. Each channel is a global stream that learns a different feature. Before this, the single vector was overburdenned, and prone to losing some knowledge as it passed through attention layers.

Notes

- The ordering of linear ops shown above is convenient for exposition; implementations often fuse projections or use learned channel-wise gates.
- Using a single compressed lane for the heavy block reduces compute and memory compared with applying the block to all lanes.

- Replace or tune the shapes (n_{hc} and d) to match your model's lane count and embedding dimension.

Quick reference

- Equation: $X_{l+1} = B_l X_l + C_l F_l(A_l X_l)$
- Typical shapes: $X_l : (n_{hc}, d)$, $A_l : (1, n_{hc})$, $F_l : (1, d) \to (1, d)$, $C_l : (n_{hc}, 1)$, $B_l : (n_{hc}, n_{hc})$

Manifold-Constrained Residual Mapping

Constraints from the paper:

$$B_l \in M := M \in R^{n \times n} | \sum_{row=1}^{n} M_{row} = 1, \sum_{col=1}^{n} M_{col} = 1, M_{element} \geq 0$$

each B_l belongs to a family of matrices M.

M is $n \times n$ matrix such that each element >= 0 and the sum of each row and each column is less than 1.

$$A_l, C_l \in A, C := R^{1 \times n}, R^{n \times 1} | A_{element} \geq 0, C_{element} \geq 0$$

Hardware activations (implementation notes):

$$A_l = \sigma(\tilde{A}_l) + \epsilon, \qquad C_l = 2 \cdot \sigma(\tilde{C}_l),$$

thus:

$$A_l, C_l \in A, C := R^{1 \times n}, R^{n \times 1} | A_{element} \leq 1, C_{element} \leq 2$$

where $\tilde{A}_l$ and $\tilde{C}_l$ are the raw, unnormalized parameters, and $\sigma(\cdot)$ is elementwise sigmoid and $\epsilon > 0$ is a small floor to avoid exact zeros in hardware. This will be explained further in Parameter Constraints and Regularization section below.

Why

- **Non_Explosiveness** $||B_l||_2 \leq 1$ **:** spectral norm of the matrix. This ensures the multiplication with B can never be increasing. Safe for forward pass and gradient backprop, avoids exploding values after 100+ of layers of transformation. Formal definition of spectral norm here: spectral norm
- **Closure Under Multiplication** $\forall M_1, M_2 \in M \Rightarrow (M_1 * M_2) \in M$ **:** This just means that as multiplication by various matrices M stack, the resulting matrice is from the set M, and has the same restrictions/properties. This ensures B is well behaved over hundreds of layers, where these transformations by B will stack many times.
- **Non-Negative Bounding** $A_{element}, C_{element}, B_{element} \geq 0$ **:** Since these matrices also stack additively, this constraint ensures they don't cause destructive interference over hundreds of layers. Without this, features could overwrite or negate each other (once more, the goal is stability across 100+ layers)
- **Bounded Magnitudes** $\sigma(\cdot) \leq 1$**:** The sigmoid function enforces strict upper boundaries. A_l is bounded near 1, and C_l is bounded at 2 (notice equation given above). This gives the network the

ability to safely scale the lanes without risking infinite amplification

- **Identity initilization $2 \cdot \sigma(\cdot)$:** Initially during training, weight matrices are initialized to small value close to zero. $\sigma(0) \approx 0.5$. Thus multiplication by 2 initializes C_l close to identity.
- **Non-Zero Floor $+\epsilon$:** A projects 4 lanes into 1 lane. If an element in A_l is ever exactly 0 (can be caused by floating point rounding, especially at low bit quantizations), the lane is disconnected from the next layers during forward pass, and previous layers during backprop. Thus we add a small off-set to ensure this doesn't occour.

Dynamic Parameterization

the parameters of A,B, and C matrices (linear mappings) are dynamically generated, using static and dynamic components.

$$\hat{X}_l = RMSNorm(vec(X_l)) \in R^{1 \times n_{hc} \cdot d}$$

where $vec(X_l) \in R^{1 \times n_{hc} \cdot d}$ is a flattening operation on $X_l \in R^{n_{hc} \times d}$

and RMSNorm is the operation:

$$RMSNorm(x) = \frac{x}{RMS(x)} = \frac{x}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \epsilon}}$$

read more here pytorch RMSNorm

$$\tilde{A}_l = \alpha_l^{pre} \cdot \hat{X}_l W_l^{pre} + S_l^{pre}$$

$$\tilde{B}_l = \alpha_l^{comb} \cdot Mat(\hat{X}_l W_l^{comb}) + S_l^{comb}$$

$$\tilde{C}_l = \alpha_l^{post} \cdot \hat{X}_l W_l^{post} + S_l^{post}$$

$$W_l^{pre}, W_l^{post} \in R^{n_{hc} \cdot d \times n_{hc}}, \qquad W_l^{comb} \in R^{n_{hc} \cdot d \times n_{hc}^2}$$

these W matrices are learnable weights for generating dynamic components.

Components

- $\tilde{A}_l, \tilde{B}_l, \tilde{C}_l$: the raw, unnormalized parameters for the A, B, and C matrices.
- $\alpha_l^{pre}, \alpha_l^{comb}, \alpha_l^{post}$: learnable scalar parameters that control the influence of the dynamic components for A, B, and C respectively.
- $S_l^{pre}, S_l^{comb}, S_l^{post}$: learnable static parameters for A, B, and C respectively.
- $W_l^{pre}, W_l^{comb}, W_l^{post}$: learnable weight matrices that project the normalized input $\hat{X}_l$ into the dynamic components for A, B, and C respectively.
- $Mat(\cdot)$: a reshaping operation that converts the output of $\hat{X}_l W_l^{comb} \in R^{1 \times n_{hc}^2}$ into $Mat(\hat{X}_l W_l^{comb}) \in R^{n_{hc} \times n_{hc}}$.

Why

- **Dynamic Components** $(\hat{X}_l W_l)$**:** these allow models to make token specific routings for each hyper-connection. For example, for a token that is a verb, the model might learn to route more information through lane 1, and for a token that is a noun, it might route more through lane 2.
- **Static Components** S_l**:** these allow the model to learn general patterns of routing that are useful across all tokens.
- **Scaling Factors** α_l**:** these allow the model to control the overall influence of the dynamic components.
- **Training Stability** $\alpha_l \approx 0$**:** the model can use static components as a base line, and organically rely on dynamic components as training progresses.
- **RMSNorm:** normalizing the input to the dynamic parameter generation helps stabilize training and ensures that the dynamic parameters are generated from a consistent scale of input features (geometrically only the direction is considered, the magnitude is normalized).
- **Flattening** $vec(X_l)$**:** this allows the dynamic parameter generation to consider interactions across all lanes and embedding dimensions, enabling more complex and informed routing decisions.
- **Reshape** $Mat(\hat{X}_l W_l^{comb})$**:** this is needed to match the shape of the B matrix, which is $n_{hc} \times n_{hc}$, allowing it to properly mix the hyper-connection lanes based on the dynamic input.

Hardware Considerations

$$W_l^{fused} \in R^{(n_{hc} \cdot d) \times (n_{hc} + n_{hc}^2 + n_{hc})}$$

- The three separate matrix multiplications for generating $\tilde{A}_l$, $\tilde{B}_l$, and $\tilde{C}_l$ can be fused into a single large matrix multiplication with a fused weight matrix $W_l^{fused} = [W_l^{pre}; W_l^{comb}; W_l^{post}]$ that concatenates the individual weight matrices. This is more efficient on GPU hardware, as it reduces the number of separate operations and allows better utilization of the GPU's parallel processing capabilities.

Parameter Constraints and Regularization

this was discussed previously, the constraints on A and B are achieved through sigmoid activations:

$$A_l = \sigma(\tilde{A}_l) + \epsilon, \qquad C_l = 2 \cdot \sigma(\tilde{C}_l)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

and the following is how to enforce the constraints we defined for the B matrix (non-explosiveness, closure under multiplication, non-negative bounding, bounded magnitudes, identity initialization):

$$M^{(0)} = Softmax(\tilde{B}_l)$$

the initial raw matrix before normalization. Softmax is applied to ensure non-negativity, one of our constraints.

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)}))$$

where $\mathscr{T}_r$ and $\mathscr{T}_c$ are the row-wise and column-wise normalization operators, respectively.

This operation diverges to $B_l = M^{(t_{max})}$, using the iterative Sinkhorn-Knopp algorithm.

in the paper $t_{max} = 20$ for practicality.

The Sinkhorn-Knopp Algorithm

Very simple.

1. Start with the raw matrix $M^{(0)}$ obtained by applying a softmax to $\tilde{B}_l$ to ensure non-negativity.
2. Iteratively normalize the matrix:

$$M_{element} = \frac{M_{element}}{\sum_{row} M_{row}} \quad (\text{i. row-wise normalization})$$

$$M_{element} = \frac{M_{element}}{\sum_{col} M_{col}} \quad (\text{ii. column-wise normalization})$$

3. After (i), matrix is normalized across rows, but not columns. after (ii), matrix is normalized across columns, but not rows. After each iteration, matrix grows closer to full constraint.
4. Usually, algorithm runs until $|\Sigma^n_{row} M_{row} - 1| \approx |\Sigma^n_{col} M_{col} - 1| \leq \Delta$ where $\Delta = 10^{-6}$, or some small threshold. But on gpu, loops with variable exit conditions are horrible for performance. So we set a fixed number of iterations, eg. 20.

Why

- **Sigmoid function σ:** The sigmoid function creates a smooth, differentiable mapping $\sigma(x) \in R | 0 \leq \sigma(x) \leq 1$ that enforces the non-negativity and upper bound constraints on A and C.